

| Question                                        | Evidence-supported answer                                                                                                                                    |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Do the main browser workflows execute?          | All 20 Chromium workflow, contract, and resilience tests passed; Firefox/WebKit and participant UAT remain incomplete.                                       |
| Are selected authorization boundaries enforced? | The five observed public/authentication/role checks returned expected statuses; this is not comprehensive security validation.                               |
| Is the runtime fully healthy?                   | Yes in the evaluated local profile: aggregate health, liveness, readiness, and the core API returned HTTP 200. This is not production monitoring validation. |
| Is performance production-ready?                | Not evaluated. Only a small sequential localhost latency sample was collected.                                                                               |

## 5.12 Threats to Validity

### 5.12.1 Construct Validity

The synthetic benchmark measures retrieval behavior for planted relevance relationships, not hiring success, candidate quality, fairness, or recruiter satisfaction. Matching metrics cannot establish whether a person should be employed. Scripted E2E success measures workflow execution, not usability or user trust.

### 5.12.2 Internal Validity

The benchmark runs inside a Spring context, so asynchronous work and persisted Matching records can affect repeatability if they are not controlled. The final test registers learning after commit, clears previous Matching state before recomputation, and checks logs for background failures. Browser tests use explicit API and UI conditions, avoid relying only on the first seeded Job, and delete the Job created by the Recruiter flow.

### 5.12.3 External Validity

The controlled data lacks organic recruiter judgments, evolving terminology, demographic diversity, adversarial behavior, and production class imbalance. The runtime is one Windows workstation with Docker Desktop, one local PostgreSQL instance, and one Chromium browser. Results cannot be generalized to enterprise load, different browsers, cloud deployment, or a population of real users.

#### **5.12.4 Reproducibility**

The dataset hash and generated JSON help reproduce the algorithm experiment. Maven Wrapper, Flyway, Testcontainers, the package lock, and recorded commands help rebuild the environment. Recruiter E2E cleanup is automatic, but exact reproduction is still harder because the working tree is not clean, the local database can change, imported records remain, and some web assets are external. The final thesis release should use a clean commit or archive together with a fixed evidence package.

#### **5.13 Chapter Summary**

The July 18 evaluation passed all 72 backend tests, completed the frontend production build, and passed all 20 Chromium workflow, contract, and resilience tests. The controlled Rocchio benchmark improved the holdout ranking on the synthetic dataset, and its final log contained no optimistic-lock exception. Aggregate health returned HTTP 200 with status UP. These results support a functioning academic prototype, but not production readiness or proven effectiveness on real recruitment data.

## CHAPTER 6. RESULTS, DISCUSSION AND CONCLUSION

### 6.1 Summary of Results

CareerFit IT AutoPilot was implemented as an IT-focused recruitment prototype with public job discovery, role-based workspaces, CV processing, matching, recommendation, Rocchio feedback, applications, AutoFit policies, email actions, analytics, and administrative monitoring. The backend follows a modular-monolith structure and the user interface is implemented in React.

The refreshed evaluation provides evidence for several technical outcomes. The complete backend suite executed 72 registered tests across 20 suites with no failures, errors, or skips. The frontend passed TypeScript checking and Vite production compilation. All 20 Chromium tests passed across role workflows, backend contracts, and resilience checks, including public search, Candidate application, Recruiter JD creation, Administrator operations, passwordless flow, CV polling, session restoration, and error states.

The controlled Rocchio benchmark showed the expected behavior on its synthetic dataset. With 50 Jobs, 100 unique CVs, 300 training pairs, and 300 holdout pairs,  $nDCG@5$  increased from 0.037737 to 0.837737,  $Recall@5$  and  $HitRate@5$  increased from 0.06 to 0.86, and  $MRR$  increased from 0.058755 to 0.842665. These results show adaptation in the designed synthetic scenario, but they do not estimate performance on real recruitment data.

The July 18 refresh retained the clean benchmark and aggregate health results, kept the largest frontend chunk below 500 kB, removed API-data fallbacks, expanded Chromium coverage to 20 tests, and refreshed the six interface screenshots. It also verified post-commit matching, Candidate-only feedback authorization, immediate match-result notifications, and portfolio privacy gating. These results support a well-tested local demonstration; they do not establish production readiness.

*Table 6.1. Consolidated result status*

| Evaluation area | Supported result | Qualification |
|-----------------|------------------|---------------|
|-----------------|------------------|---------------|

| Evaluation area               | Supported result                                                          | Qualification                                                                                 |
|-------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Backend automated tests       | 72/72 registered tests passed                                             | Final logs were also checked for background exceptions                                        |
| Controlled feedback benchmark | Large, deterministic improvement after Rocchio                            | Synthetic causal design; not production effectiveness                                         |
| Frontend build                | TypeScript and Vite build passed                                          | Largest JavaScript chunk was 378.44 kB; no Vite size warning                                  |
| Browser workflows             | 20/20 Chromium tests passed                                               | Workflow, contract, and resilience coverage; no Firefox/WebKit or independent participant UAT |
| Authorization spot checks     | Observed 200/401/403 outcomes matched expectations                        | Not a complete security assessment                                                            |
| Runtime APIs                  | Core Job API and aggregate/liveness/readiness health returned HTTP 200/UP | Local profile only; no external production monitoring                                         |
| Local latency sample          | Mean 61.79 ms and p95 85.11 ms for 30 sequential Job queries              | No concurrency, controlled load, or production network                                        |

## 6.2 Achievement of Thesis Objectives

### 6.2.1 Role-Based Recruitment Platform

The objective of providing role-specific workflows was achieved at prototype level. Guests can browse public Jobs and employer information. Candidates can maintain profiles, CVs, and portfolios; inspect matching and recommendation results; apply or withdraw; submit feedback; and configure automation. Recruiters can manage Jobs, inspect applicants and discovered candidates, invite candidates, and update application status. Administrators can monitor users, Jobs, audit records, and email/token state. The Chromium P0 suite confirms representative workflows for all four interface roles, although it does not cover every route or browser.

### **6.2.2 CV and Job-Description Processing**

The project implements uploaded and manually created CVs, explicit processing statuses, file-type and magic-byte validation, PDF and DOCX extraction, image and scanned-PDF OCR fallback, sparse-text warning, failure reason persistence, and structured quality signals. Job creation includes structured fields and conditional validation. This objective is supported by source inspection and automated tests for extraction, validation, Job service behavior, and integration contracts.

The implementation is limited by local storage, the availability of external Tesseract software, page and timeout limits, and a simple deterministic tokenizer. It should be described as a working ingestion pipeline rather than a general document-understanding system.

### **6.2.3 Matching and Recommendation**

CareerFit implements a static-corpus TF-IDF vectorizer, cosine similarity, normalized score, LOW/MEDIUM/HIGH labels, potential heuristics, and shared-term reasons. CV-JD matching and profile-oriented recommendation remain separate workflows, and Matching state remains separate from Application state. Scoring, TF-IDF, batch isolation, and API contract tests passed.

The objective was achieved as an interpretable lexical baseline, not as a semantic matching model. Technology names with punctuation, synonyms, Vietnamese phrases, career changes, and context beyond shared terms remain limitations. The displayed percentage is a normalized similarity score, not a tested probability of recruitment success.

### **6.2.4 Feedback Learning**

The system records GOOD\_MATCH, POTENTIAL, BAD\_MATCH, and NOT\_INTERESTED feedback with actor and channel information. Positive and negative learning signals update the Job representation using Rocchio with  $\alpha=1.0$ ,  $\beta=0.75$ , and  $\gamma=0.15$ . Recalculation begins from the original Job vector and current feedback history, which produces repeatable results for the same inputs. Affected Matchings are marked stale and rescored asynchronously.

The controlled benchmark confirms the intended Rocchio behavior in the planted latent-skill scenario. The final run completed without the earlier optimistic-lock exception after learning was moved to an after-commit boundary and benchmark

state was cleared before recomputation.  $nDCG@5$  and  $HitRate@5$  each increased by 0.80, although this does not prove reliability under production concurrency.

### 6.2.5 Policy-Driven Automation and Email Action

AutoFit policies store thresholds, enablement, notification preferences, cooldown, quota, quiet-hour, timezone, digest, and pause settings. Scheduled services recompute stale results, send digests and high-match notifications, clean expired email actions, and execute auto-apply. Auto-apply validates the default CV, Job status, threshold, and duplicate state and limits creation to three applications per run.

This objective is achieved as configurable prototype automation. Notification policy and application authorization remain separate control paths, scheduler timing is externalized through `app.scheduler.*`, and the email-action channel uses hashed token storage with GET confirmation followed by POST execution. Further production work is still required for rate limiting, secret rotation, mail delivery, and deployment-specific origin controls.

### 6.2.6 Auditability and Evaluation

Major application, feedback, automation, and administrative actions create structured audit rows. Notification delivery and email-action statuses add operational evidence. The project includes unit, integration, security, algorithm, and E2E tests and produces a dataset-hashed benchmark artifact.

Audit coverage has not been formally proven for every state-changing endpoint, and direct repository calls can still produce inconsistent metadata conventions. The evaluation nevertheless checked logs and side effects in addition to test exit codes, which helped detect and correct the earlier concurrency and health problems.

*Table 6.2. Objective assessment*

| Objective                                   | Assessment                                        | Primary evidence                                                       |
|---------------------------------------------|---------------------------------------------------|------------------------------------------------------------------------|
| Role-based web platform                     | Achieved for prototype scope                      | API contracts and 20 Chromium workflow, contract, and resilience tests |
| CV/JD ingestion and validation              | Achieved for supported formats and configured OCR | Extraction, validation, Job, and integration tests                     |
| Explainable lexical matching/recommendation | Achieved as baseline                              | TF-IDF/scoring tests and implemented reasons                           |

| Objective                            | Assessment                                                         | Primary evidence                                                                                 |
|--------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Rocchio feedback adaptation          | Achieved in the controlled test; production concurrency unverified | Current synthetic benchmark, after-commit execution, and clean background log                    |
| AutoFit and actionable communication | Achieved as a configurable prototype                               | Auto-apply tests, scheduler, immediate notifications, and hashed confirm-then-POST email actions |
| Security and auditability            | Partially verified                                                 | Security tests and spot checks; incomplete penetration/audit coverage                            |
| Production readiness                 | Not demonstrated                                                   | Local health passed; scale, independent users, and production security evidence remain missing   |

## 6.3 Discussion

### 6.3.1 Value of an Interpretable Baseline

CareerFit uses a lexical vector-space baseline instead of assuming that a dense or generative model is always better. This choice lets users inspect the input tokens, weights, shared terms, feedback centroids, and score calculation. For an academic system, this transparency helps explanation, debugging, boundary testing, and presentation of the implementation.

The cost is reduced semantic capability. A model that cannot reliably equate related technologies or interpret career context may under-rank suitable candidates and over-rank documents containing repeated keywords. The correct conclusion is not that TF-IDF is sufficient for all recruitment, but that it provides a reproducible baseline against which future semantic or hybrid retrieval can be compared.

### **6.3.2 Human-in-the-Loop as System Structure**

Human-in-the-Loop in CareerFit is expressed through multiple control points: users supply and validate data, inspect reasons, configure policy, apply or invite, provide feedback, pause automation, and review audit history. The system distinguishes score evidence from Application state and policy action. This gives users more control than adding a general approval button after an automated decision that they cannot understand.

However, human oversight does not automatically make a system fair or safe. Users may accept misleading explanations, policies may use unsuitable thresholds, and feedback may repeat individual or organizational bias. HITL must therefore be combined with error analysis, access control, ways to question or appeal results, audit review, and evaluation with representative data.

### **6.3.3 Meaning of the Rocchio Improvement**

The large benchmark delta is expected from the experimental design: a latent skill is deliberately shared between a training-positive CV and a holdout-positive CV but omitted from the initial JD emphasis. Rocchio moves the Job vector toward the training example, making the holdout example easier to retrieve. Repeated equality of the metrics shows implementation determinism under this scenario.

The result does not show that every real Candidate feedback event improves ranking by 0.80 nDCG@5. Organic feedback may be inconsistent, sparse, biased, delayed, or based on salary and location rather than technical relevance. Production evaluation would require independently labeled data, time-based splits, multiple reviewers, disagreement analysis, and online or offline comparison against a baseline without feedback.

### **6.3.4 Green Assertions versus Operational Cleanliness**

The earlier background `StaleObjectStateException` showed why test counts cannot be the only release check. JUnit assertions can pass while asynchronous tasks log failures outside the main test flow. Component health and overall health must also be read together. A reliable system report should check logs, health components, output files, and side effects in addition to exit codes.

Future CI should continue to fail on uncaught background exceptions, control schedulers during algorithm tests, wait for asynchronous work explicitly, and archive

logs as first-class results. Operational health details should be protected while remaining available to authorized operators.

### **6.3.5 Product Positioning**

CareerFit should be positioned as an IT-focused, explainable, policy-controlled academic recruitment prototype. It demonstrates how Job portal functions, ranking, recommendation, feedback, automation, email actions, and audit can be connected. It should not be positioned as having better general-purpose AI than commercial recruiter platforms or as replacing professional recruitment judgment.

The clearest difference is workflow control: matching and recommendation are separate, policies are explicit, users keep control points, feedback has a visible mathematical effect, and actions can be audited. This difference is limited in scope but is well supported by the implementation.

## **6.4 Limitations**

### **6.4.1 Data and Algorithm Limitations**

The controlled dataset is synthetic and designed to show how feedback changes ranking. It does not include natural language variety, recruiter disagreement, demographic analysis, intentionally misleading CVs, or changing labor-market patterns. The runtime database contains seed and scraped Jobs but no validated relevance labels. TF-IDF uses a manually created static IT corpus, whitespace tokenization, and maximum IDF for unknown terms. Potential detection uses fixed shared-term and seniority rules.

### **6.4.2 Evaluation Limitations**

The 72 backend tests do not prove exhaustive path coverage. Browser evaluation covers 20 automated cases in Chromium only, and no independent users participated. Security checks were targeted API observations rather than penetration testing, while the latency sample used 30 sequential warm requests on one workstation. Aggregate health passed in the final run, but concurrency, capacity, cross-browser behavior, and real-user usability remain unevaluated.

### 6.4.3 Security and Privacy Limitations

Notification action tokens are hashed, and state changes require POST after confirmation. Access tokens are stored in frontend sessionStorage rather than persistent localStorage. Local CV storage lacks documented malware scanning, encryption, retention enforcement, and recovery testing. Public Swagger and Prometheus access are suitable for local demonstration but should be restricted in production. Privacy, fairness, and management of users' personal data have not been validated with a real deployment population.

### 6.4.4 Reliability and Maintainability Limitations

The final remediation pass resolved the observed optimistic-lock benchmark exception, scheduler and configuration drift, mock Job fallback values, persistent localStorage authentication, the oversized frontend bundle warning, and missing E2E Job cleanup. Audit records are still written directly by multiple services, and the evaluated working tree is not a fixed release commit. These limits make production operation and exact reproduction of the results more difficult.

*Table 6.3. Principal limitations and impact*

| <b>Limitation</b>                                       | <b>Impact on conclusions</b>                                                                    |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Synthetic benchmark                                     | Supports causal behavior only, not real hiring effectiveness                                    |
| Lexical representation                                  | Limits synonym, semantic, and context understanding                                             |
| Production concurrency not evaluated                    | Final benchmark is clean, but production conflict and retry behavior remain unproven            |
| Local-only health verification                          | Health passed locally but does not establish deployed availability                              |
| Chromium-only 20-test automation suite                  | Limits cross-browser and real-user usability conclusions                                        |
| Email delivery and abuse controls not production-tested | Hashed confirm-then-POST flow is implemented; rate limiting and real delivery remain unverified |
| Small localhost latency sample                          | Cannot establish scale, throughput, or cloud performance                                        |
| Dirty worktree and mutable E2E database                 | Weakens exact experiment reproduction                                                           |